ML Models For Hate Speech Detection

According to the base paper “Offensive Language Detection in Nepali Social Media” , they used various Machine Learning approaches and Transformers in their paper , in order to replicate the paper results , I plan to use the following approaches :

- Logistic Regression
- Simple Vector Machine
- Random Forest
- M-BERT

Date Pre-processing

The labels on data and their meaning are all defined in the paper itself . But in brief there are two types :

- Languages → Romanized Nepali and Devnagari Nepali
- Classes → Label_Binary and Label_Multiclass

Sample JSON Data


```
    {
        "ID": "vumtps",
        "Comment": "आज जीउदै छौ कसलाई के थाहा छ र भोली जीउदै भईन्छ की भईदैन मरीलानू के छ र एक जूनीमा😃सकेसम्म राम्रो बन्न सीक😃सीके सीक नसीके न सीक😉😋😅😂",
        "Label_Binary": "NOFF",
        "Label_Multiclass": "NO"
    }
```


Further Pre-processing

In the paper several pre-processing steps have already been implemented but further more we can include steps like :

- Keeping the emoji in the sentences which can preserve satire .
- Keeping punctuation marks .

```
        'ि': 'ी',
         'ु': 'ू',
         'े': 'ै',
         'ो': 'ो',
         'क्ष': 'क्ष'
```

As throughout the paper the dataset were trained and tested in Binary Classification , so for time being , we will be setting Label_Binary as category.

```
# Update these paths to your actual file names
train_file = <file_path>
test_file  = <file_path>

def load_json_df(file_path):
    return pd.read_json(file_path, encoding='utf-8')

# Load
train_df = load_json_df(train_file)
test_df  = load_json_df(test_file)

print(f"Train size: {len(train_df)}, Test size: {len(test_d
f)}")
print("\nTrain label distribution:")

print(train_df['Label_Binary'].value_counts())
print("\nTest label distribution:")
print(test_df['Label_Binary'].value_counts())
```

```
Train size: 5798, Test size: 1450

Train label distribution:
Label_Binary
NOFF    3562
OFF     2236
Name: count, dtype: int64
```

```
Test label distribution:
Label_Binary
NOFF    896
OFF     554
Name: count, dtype: int64
```

Though most of the cleaning and normalisation process is already applied in the dataset , we process it through a simple cleaning function to remove irregularities or any thing that was missed.

```
import re
import unicodedata
from indic_transliteration import sanscript

# Step 1: Basic cleaning
def clean_text(text):
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'\s+', ' ', text)
    return text.strip().lower()

# Step 2: Dirghikaran + normalization
def dirghikaran(text):
    text = unicodedata.normalize('NFC', text)
    replacements = {
        'ि': 'ी',
        'ु': 'ू',
        'े': 'ै',
        'ो': 'ौ',
        'क्ष': 'क्ष'
    }
    for short, long_form in replacements.items():
        text = text.replace(short, long_form)
    return text

# Step 3: Romanization [No need rominization ,keep nepali i
n nepali , english in english]
# def romanize(text):
#     return sanscript.transliterate(text, sanscript.DEVANA
```

```
GARI, sanscript.IAST)

# Step 4: Full preprocessing pipeline
def preprocess_text_full(text):
    text = clean_text(text)
    text = dirghikaran(text)
    #text = romanize(text)
    return text


# Apply to both
train_df['clean_text'] = train_df['Comment'].apply(preproce
ss_text_full)
test_df['clean_text']  = test_df['Comment'].apply(preproces
s_text_full)
# Check a few examples
print(train_df[['Comment', 'clean_text', 'Label_Multiclas
s']].head(10)
```

Hence the data will be pre-processed as ,(all the grammar has been normalized तिर → तीर) :

```
3  मान्छे मात्र होइन गाउँ तिर भैसी ब्याउने समय पन...

Gets Processed to

3  मान्छे मात्र होइन गाउँ तीर भैसी ब्याउने समय पन...
```

Liner Regression

In the paper they have tested on many conditions ,among them they had concluded :

- The results in obtained from training and testing “characters only” wise
- Combining texts using both Romanization and Dirghikaran gave better result.

That means the best data for evaluation from the table (Table-5) is below is the 8th row of data.

Prep.	Non-Offensive			Offensive		
	Pre.	Rec.	F_1	Pre.	Rec.	F_1
A	0.71	0.94	0.81	0.80	0.39	0.52
B	0.71	0.94	0.81	0.81	0.39	0.53
C	0.76	0.94	0.84	0.85	0.51	0.64
D	0.76	0.95	0.84	0.85	0.51	0.64
A	0.78	0.94	0.85	0.84	0.56	0.68
B	0.78	0.92	0.85	0.83	0.58	0.68
C	0.79	0.91	0.84	0.81	0.60	0.69
D	0.79	0.92	0.85	0.83	0.59	0.69
A	0.78	0.93	0.85	0.83	0.57	0.68
B	0.79	0.92	0.85	0.83	0.60	0.69
C	0.79	0.92	0.85	0.81	0.60	0.69
D	0.79	0.93	0.85	0.83	0.61	0.70

Table 5: Effect of prepossessing techniques and features on binary classification. Preprocessing techniques: (A) No Preprocessing (Prep_None) (B) Dirghikaran (Prep_Dir), (C) Romanization (Prep_Rom), and (D) Prep_Dir + Prep_Rom. The **first block** uses word only, the **second block** uses character only and the **last block** uses both word and character features.

The paper then claims to have performed training on different models ,with the criteria as follows:

- Reduced the features using KBest to 10000 for both train and test data sets.
- Logistic Regression (LR): Linear LR with L2 regularization constant 1 and limited-memory BFGS optimization.
- Support Vector Machine (SVM): Linear SVM with L2 regularization constant 1 and logistic loss function.
- Random Forests (RF): Averaging probabilistic predictions of 100 randomized decision trees.
- Multilingual BERT (MBERT)

Which obtained the values as follows :

Models	Non-Offensive			Offensive		
	Pre.	Rec.	F_1	Pre.	Rec.	F_1
Baseline	0.74	0.73	0.73	0.57	0.58	0.58
LR	0.80	0.93	0.87	0.86	0.63	0.72
SVM	0.78	0.95	0.85	0.87	0.56	0.68
RF	0.81	0.93	0.86	0.85	0.64	0.73
M-BERT	0.74	0.90	0.81	0.75	0.49	0.59

Table 6: Binary classification using different machine learning models.

Logistic Regression

When applied similar methods as mentioned in the paper

```
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report  # For de
tailed pre/rec/f1


X_train = train_df['clean_text']
y_train = train_df['Label_Binary']

X_test = test_df['clean_text']
y_test = test_df['Label_Binary']

# Pipeline with chi-squared feature selection (k=10000 as i
n the paper)
pipeline_chi2 = Pipeline([
    ('tfidf', TfidfVectorizer(
        analyzer='char',
        ngram_range=(3, 3),     #paper has mentioned charac
ter tri-gram
```

```
        max_features=None,     # No limit here — let chi2 s
elect the best (or set high, e.g., 200000)
        lowercase=True
    )),
    ('feature_selection', SelectKBest(chi2, k=10000)),  # K
ey step: top 10,000 by chi-squared
    ('clf', LogisticRegression(
        max_iter=1000,
        class_weight='balanced',
        solver='lbfgs',
        random_state=42
    ))
])

pipeline_chi2.fit(X_train, y_train)
preds = pipeline_chi2.predict(X_test)

# Print detailed metrics (pre=precision, rec=recall, f1)
print("=== TF-IDF + Chi2 SelectKBest(10000) + Logistic Regr
ession ===")
print(classification_report(y_test, preds, digits=4))  # Fu
ll per-class + macro avg

# Also get Macro F1 for comparison
from sklearn.metrics import f1_score
print("Macro F1:", f1_score(y_test, preds, average='macr
o'))
```

```
=== TF-IDF + Chi2 SelectKBest(10000) + Logistic Regression ===
              precision    recall  f1-score   support

        NOFF     0.8337    0.8281    0.8309       896
         OFF     0.7250    0.7329    0.7289       554

    accuracy                         0.7917      1450
   macro avg     0.7794    0.7805    0.7799      1450
weighted avg     0.7922    0.7917    0.7919      1450
```

```
Macro F1: 0.7799059511339945
```

SVM

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC  # Linear SVM — fast and
strong for text
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, f1_scor
e, accuracy_score

X_train = train_df['clean_text']
y_train = train_df['Label_Binary']

X_test = test_df['clean_text']
y_test = test_df['Label_Binary']

# 1. Standard Linear SVM (squared_hinge loss, L2 reg, C=1)
svm_hinge = Pipeline([
    ('tfidf', TfidfVectorizer(
        analyzer='char',
        ngram_range=(3, 6),   # Char n-grams capture Nepali
subwords well
        max_features=None
    )),
    ('feature_selection', SelectKBest(chi2, k=10000)),  # O
ptional but helps
    ('clf', LinearSVC(
        class_weight='balanced',
        max_iter=1000,
        random_state=42
    ))
])

svm_hinge.fit(X_train, y_train)
```

```
preds_svm = svm_hinge.predict(X_test)

print("=== Linear SVM  ===")
print(classification_report(y_test, preds_svm, digits=4))
print("Macro F1:", f1_score(y_test, preds_svm, average='mac
ro'))
```

```
=== Linear SVM  ===
              precision    recall  f1-score   support

        NOFF     0.8381    0.8549    0.8464       896
         OFF     0.7575    0.7329    0.7450       554

    accuracy                         0.8083      1450
   macro avg     0.7978    0.7939    0.7957      1450
weighted avg     0.8073    0.8083    0.8076      1450

Macro F1: 0.7956814841096862
```

Random Forest [Binary Classification]

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, f1_scor
e, accuracy_score


X_train = train_df['clean_text']
y_train = train_df['Label_Binary']

X_test = test_df['clean_text']
y_test = test_df['Label_Binary']

# Random Forest Pipeline
rf_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        analyzer='char',
```

```
        ngram_range=(3, 6),
        max_features=30000  # Important: limits features fo
r faster training
    )),
    ('clf', RandomForestClassifier(
        n_estimators=200,          # 100-300 trees; 200 is
a good balance
    ))
])

rf_pipeline.fit(X_train, y_train)
rf_preds = rf_pipeline.predict(X_test)

print("=== Random Forest Results ===")
print("Accuracy:", accuracy_score(y_test, rf_preds))
print("Macro F1:", f1_score(y_test, rf_preds, average='macr
o'))
print("\nFull Classification Report:")
print(classification_report(y_test, rf_preds))
```

```
=== Random Forest Results ===
Accuracy: 0.8227586206896552
Macro F1: 0.8061008130081301

Full Classification Report:
              precision    recall  f1-score   support

       NOFF       0.83      0.90      0.86       896
        OFF       0.82      0.69      0.75       554

    accuracy                           0.82      1450
   macro avg       0.82      0.80      0.81      1450
weighted avg       0.82      0.82      0.82      1450
```

Random Forest [Multiclass Classification]

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, f1_scor
```

```
e, accuracy_score
import numpy as np
import pandas as pd

# === Per Paper: Hierarchical Classification ===
# Step 1: Train binary classifier (Offensive vs Non-Offensi
ve)
print("=== Step 1: Training Binary Classifier (OFF vs NOFF)
===")

X_train = train_df['clean_text']
y_train_binary = train_df['Label_Binary']   # 'NOFF' / 'OF
F'

X_test = test_df['clean_text']
y_test_multiclass = test_df['Label_Multiclass']  # Ground t
ruth: NO, OO, OR, OS

binary_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        analyzer='char',
        ngram_range=(3, 6),
        max_features=30000
    )),
    ('clf', RandomForestClassifier(
        n_estimators=300,
        class_weight='balanced',
        random_state=42,
        n_jobs=-1
    ))
])

binary_pipeline.fit(X_train, y_train_binary)
binary_preds = binary_pipeline.predict(X_test)
print(f"Binary classifier trained. Predictions: {np.unique
(binary_preds)}\n")

# === Step 2: Train fine-grained classifier on ONLY offensi
```

```
ve samples ===
print("=== Step 2: Training Fine-Grained Classifier (OO vs
OR vs OS) ===")

# Filter training data: only offensive samples
train_offensive = train_df[train_df['Label_Binary'] == 'OF
F'].copy()
X_train_fine = train_offensive['clean_text']
y_train_fine = train_offensive['Label_Multiclass']  # Only
OO, OR, OS (no NO)

print(f"Fine-grained training set size: {len(train_offensiv
e)}")
print(f"Classes in fine-grained set: {np.unique(y_train_fin
e)}\n")

fine_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        analyzer='char',
        ngram_range=(3, 6),
        max_features=30000
    )),
    ('clf', RandomForestClassifier(
        n_estimators=300,
        class_weight='balanced',   # Important: OR/OS are r
are
        random_state=42,
        n_jobs=-1
    ))
])

fine_pipeline.fit(X_train_fine, y_train_fine)
print("Fine-grained classifier trained.\n")

# === Step 3: Combine predictions (hierarchical inference)
===
print("=== Step 3: Hierarchical Prediction on Test Set ==
=")
```

```
final_preds = []

for i, text in enumerate(X_test):
    # First classifier decision
    binary_pred = binary_preds[i]

    if binary_pred == 'NOFF':
        # Non-offensive: assign NO
        final_preds.append('NO')
    else:  # binary_pred == 'OFF'
        # Offensive: use fine-grained classifier to determi
ne OO/OR/OS
        fine_pred = fine_pipeline.predict([text])[0]
        final_preds.append(fine_pred)

final_preds = np.array(final_preds)

print(f"Predictions distribution: {np.unique(final_preds, r
eturn_counts=True)}\n")

# === Step 4: Evaluate on full 4-class task (NO + OO + OR +
OS) ===
print("=" * 70)
print("=== HIERARCHICAL RANDOM FOREST RESULTS (Paper Method
ology) ===")
print("=" * 70)
print(f"Accuracy: {accuracy_score(y_test_multiclass, final_
preds):.4f}")
print(f"Macro F1: {f1_score(y_test_multiclass, final_preds,
average='macro'):.4f}")
print(f"Weighted F1: {f1_score(y_test_multiclass, final_pre
ds, average='weighted'):.4f}")

print("\nFull Classification Report:")
print(classification_report(y_test_multiclass, final_preds,
digits=4))
```

```python
# Optional: Per-class performance
report_df = pd.DataFrame(
    classification_report(y_test_multiclass, final_preds, output_dict=True)
).T
print("\nPer-Class Metrics:")
print(report_df[['precision', 'recall', 'f1-score']].round(4))
```

```
=== Step 1: Training Binary Classifier (OFF vs NOFF) ===
Binary classifier trained. Predictions: ['NOFF' 'OFF']

=== Step 2: Training Fine-Grained Classifier (OO vs OR vs OS) ===
Fine-grained training set size: 2236
Classes in fine-grained set: ['OO' 'OR' 'OS']

Fine-grained classifier trained.

=== Step 3: Hierarchical Prediction on Test Set ===
Predictions distribution: (array(['NO', 'OO', 'OR'], dtype='<U2'), array([975, 436,  39]))

======================================================================
=== HIERARCHICAL RANDOM FOREST RESULTS (Paper Methodology) ===
======================================================================
Accuracy: 0.8048
Macro F1: 0.5331
Weighted F1: 0.7955

Full Classification Report:
              precision    recall  f1-score   support

          NO     0.8318    0.9051    0.8669       896
          OO     0.7615    0.6831    0.7202       486
          OR     0.6154    0.4898    0.5455        49
          OS     0.0000    0.0000    0.0000        19

    accuracy                         0.8048      1450
   macro avg     0.5522    0.5195    0.5331      1450
weighted avg     0.7900    0.8048    0.7955      1450


Per-Class Metrics:
              precision  recall  f1-score
NO               0.8318  0.9051    0.8669
OO               0.7615  0.6831    0.7202
OR               0.6154  0.4898    0.5455
OS               0.0000  0.0000    0.0000
accuracy         0.8048  0.8048    0.8048
```

```
macro avg        0.5522  0.5195    0.5331
weighted avg     0.7900  0.8048    0.7955
```